

脑韵科技 Naoyun SDK Python 版用户接口 手册

Table of Contents

使用说明	3
购买 SDK	3
常见问题	3
1. 快速开始	4
1.1 环境要求	4
1.2 安装与引用	4
1.3 最小使用示例	4
2. 核心类与接口	6
2.1 `NaoyunSdkApi` — SDK 主入口	6
2.2 算法任务	8
2.3 回调事件	8
3. 数据模型	9
3.1 `BleDeviceInfo`	9
3.2 `DeviceStateEventArgs`	9
3.3 `DeviceStatusNotificationEventArgs`	10
3.4 `DataReceivedEventArgs`	10
3.5 `SpectrumDataEventArgs`	11

3.6 `MentalStateDataEventArgs`	11
3.7 `ServerAuthResultEventArgs`	12
4. 枚举定义	12
4.1 `ConnectionType`	12
4.2 `EarSide`	12
4.3 `SignalQuality`	13
5. 辅助函数	13
5.1 `save_eeg_to_excel`	13
6. 完整使用示例	14
6.1 搜索、连接与原始数据接收	14
6.2 启动脑状态任务	14
6.3 获取滤波后的 EEG 数据并保存	15

版本: PythonSDK_1.0.1.0

适用平台: Windows (x64)

使用说明

- 由于是 ble 传输, 本产品不适用实时性要求较高的 ERP 实验。

购买 SDK

- 获取 applID 和 Secret, 联系: contact@brainrhythm.cn or support@veetra.ai。
- 下载 sample: <https://github.com/brainrhythm>, 目前有 python 和 C# 版本。
- python SDK 版本下载链接: https://github.com/brainrhythm/Naoyun_SDK_Python
- C# SDK 版本下载链接: https://github.com/brainrhythm/Naoyun_SDK_Csharp
- 验证订单, 通过邮件发送 applID 和 Secret。

常见问题

1. scan 的时候搜索不到设备, 排查原因:

- 确保耳机已经开启, 或者放回耳机仓灯亮后, 3 秒后再拿出;
- PC 端/笔记本蓝牙设备是否支持 BLE 功能;
- 耳机 BLE 是不是被其他设备连接, 手机上脑韵/Veetra 从后台杀掉后再去搜索;

2. python 版本中, 需保证库文件和 naoyundemo.py 在同一目录, 在 naoyundemo.py 所在目录中执行,

- 如: `python naoyundemo.py ak_xxxxxx sdk_xxxxxx`

1. 快速开始

1.1 环境要求

- Python 3.13 或更高版本（64 位）
- 操作系统：Windows 10/11 x64
- 依赖库（按需安装）：
 - `bleak` — BLE 蓝牙功能
 - `numpy` — 算法与数据处理
 - `scipy` — 带通滤波
 - `aiohttp` — 服务器异步认证
 - `requests` — 服务器同步认证
 - `openpyxl` — Excel 文件保存
 - `matplotlib` — 数据可视化

1.2 安装与引用

将发行包中的以下文件放置到项目目录：

- `naoyunsdk.cp313-win_amd64.pyd`（或对应 Python 版本的 `.pyd`）
- `libmath.dll`

在代码中直接引用：

```
import naoyunsdk
from naoyunsdk import (
    NaoyunSdkApi, ServerAuthClient,
    ConnectionType, EarSide, SignalQuality,
    BleDeviceInfo, DeviceStateEventArgs,
    DeviceStatusNotificationEventArgs,
    DataReceivedEventArgs, SpectrumDataEventArgs,
    MentalStateDataEventArgs, ServerAuthResultEventArgs,
    save_eeg_to_excel
)
```

1.3 最小使用示例

```
import asyncio
import datetime
from naoyunsdk import NaoyunSdkApi, EarSide
```

```
async def main():
    sdk = NaoyunSdkApi()

    # 1. 初始化（填入你的 AppID 和 Secret）
    sdk.Initialize(app_id="YOUR_APP_ID", app_secret="YOUR_APP_SECRET", is_domestic=True)

    # 2. 搜索设备（5 秒）
    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if not devices:
        print("未找到设备")
        return

    # 3. 连接第一个设备
    connected = await sdk.ConnectBleAsync(devices[0].Id)
    if not connected:
        print("连接失败")
        return

    # 4. 开始采集数据
    await sdk.SendStartDataAsync()

    # 5. 采集 10 秒
    await asyncio.sleep(10)

    # 6. 获取最近 5 秒 EEG 数据
    left_data = sdk.GetLatestEegData(EarSide.Left, seconds=5)
    right_data = sdk.GetLatestEegData(EarSide.Right, seconds=5)

    # 7. 保存到 Excel
    save_eeg_to_excel(left_data, right_data)

    # 8. 停止并断开
    await sdk.SendStopDataAsync()
    await sdk.DisconnectAsync()

asyncio.run(main())
```

2. 核心类与接口

2.1 `NaoyunSdkApi` — SDK 主入口

初始化与版本

方法	说明
<code>Initialize(app_id, app_secret, is_domestic=True)</code>	初始化 SDK，设置服务器认证信息。 <code>is_domestic=True</code> 使用国内服务器。
<code>GetSdkVersion()</code>	返回 SDK 版本字符串。

BLE 设备搜索

方法	说明
<code>async StartBleScanAsync(timeout)</code>	搜索 BLE 设备， <code>timeout</code> 为 <code>datetime.timedelta</code> 类型。搜索到的设备会通过 <code>BleDeviceDiscovered</code> 回调返回，方法结束时也返回设备列表 <code>List[BleDeviceInfo]</code> 。
<code>async StopBleScanAsync()</code>	停止正在进行的 BLE 搜索。

连接与断开

方法	说明
<code>async ConnectBleAsync(bluetooth_address)</code>	连接指定 BLE 设备（传入设备地址字符串）。连接成功后会自动获取设备状态，若已调用 <code>Initialize</code> 还会自动进行服务器认证。
<code>async DisconnectAsync()</code>	断开当前连接。

数据采集控制

方法	说明
<code>async SendStartDataAsync()</code>	发送开始数据采集命令。
<code>async SendStopDataAsync()</code>	发送停止数据采集命令。

<code>async SendInitCommandAsync()</code>	发送初始化命令，获取设备完整状态。
<code>async GetMacAddressAsync()</code>	获取设备 MAC 地址。获取结果可通过 <code>MacAddress</code> 属性读取。

设备状态控制

方法	说明
<code>async SetNoiseReductionModeAsync(mode)</code>	设置降噪模式。 <code>mode</code> : 0=普通, 1=降噪, 2=环境音。返回 (发送成功, 验证成功)。
<code>async SetTouchEnabledAsync(enabled)</code>	设置触摸开关。返回 (发送成功, 验证成功)。
<code>async SetAutoPlayEnabledAsync(enabled)</code>	设置智能播放 (佩戴检测自动播放)。返回 (发送成功, 验证成功)。

数据读取

方法	说明
<code>GetLatestEegData(ear_side, seconds, use_filtered_data=False)</code>	获取最近 N 秒的 EEG 数据。 <code>ear_side</code> 取值为 <code>EarSide.Left</code> 或 <code>EarSide.Right</code> ; <code>seconds</code> 范围 0~60; <code>use_filtered_data=True</code> 时返回 1-78Hz 带通滤波后的数据。
<code>get_current_status()</code>	获取设备当前状态字典, 包含电量、佩戴状态、降噪模式、触摸开关等信息。

属性

属性	说明
<code>IsConnected</code>	<code>bool</code> , 当前是否已连接设备。
<code>MacAddress</code>	<code>str</code> 或 <code>None</code> , 最近一次读取到的设备 MAC 地址 (大写十六进制字符串, 无分隔符)。

2.2 算法任务

能量谱分析

方法	说明
<code>StartSpectrumTask()</code>	启动能量谱后台分析任务。分析结果通过 <code>SpectrumDataReceived</code> 回调实时反馈。
<code>StopSpectrumTask()</code>	停止能量谱任务。

实时脑状态

方法	说明
<code>StartMentalStateTask(interval_seconds=1.0)</code>	启动实时脑状态反馈任务。通过 <code>MentalStateDataReceived</code> 回调定时反馈专注度、疲劳度等指标。 <code>interval_seconds</code> 范围 0.5~5.0 秒。
<code>StopMentalStateTask()</code>	停止脑状态任务。

2.3 回调事件

所有回调均为可赋值属性，支持普通函数与 `async` 协程函数。

回调名称	参数类型	触发时机
<code>BleDeviceDiscovered</code>	<code>BleDeviceInfo</code>	搜索到 BLE 设备时
<code>ConnectionStateChanged</code>	<code>DeviceStateEventArgs</code>	连接状态变化时
<code>DeviceStatusNotificationReceived</code>	<code>DeviceStatusNotificationEventArgs</code>	收到设备状态通知时
<code>DataReceived</code>	<code>DataReceivedEventArgs</code>	收到 EEG 原始数据时
<code>SpectrumDataReceived</code>	<code>SpectrumDataEventArgs</code>	能量谱分析产生新数据时
<code>SpectrumTaskStatusChanged</code>	<code>(bool, str)</code>	能量谱或脑状态任务启停时（参数: <code>isRunning</code> , <code>message</code> ）
<code>MentalStateDataReceived</code>	<code>MentalStateDataEventArgs</code>	脑状态分析产生新数据时

ServerAuthCompleted	ServerAuthResultEventArgs	服务器认证完成时
on_log_message	str	SDK 内部产生日志消息时

回调设置示例：

```
def on_device_found(device: BleDeviceInfo):  
    print(f"发现设备: {device.Name} [{device.Id}]")  
  
sdk.BleDeviceDiscovered = on_device_found
```

3. 数据模型

3.1 `BleDeviceInfo`

字段	类型	说明
Id	str	设备地址/标识符
Name	str	设备名称
BluetoothAddress	int	蓝牙地址（数值形式）
IsConnected	bool	是否已连接（默认值 <code>False</code> ）

3.2 `DeviceStateEventArgs`

字段	类型	说明
ConnectionType	ConnectionType	连接类型（目前仅 <code>BLE</code> ）
IsConnected	bool	是否已连接
Message	str	状态描述信息

3.3 `DeviceStatusNotificationEventArgs`

字段	类型	说明
Command	int	命令码
EarSide	int	耳侧标识
LeftBattery	int	左耳电量 (0~100)
RightBattery	int	右耳电量 (0~100)
LeftWorn	bool	左耳是否佩戴
RightWorn	bool	右耳是否佩戴
HardwareVersion	int	硬件版本号
SoftwareVersion	int	软件版本号
IsBigEndian	bool	是否大端序
NoiseReductionMode	int	当前降噪模式
TouchEnabled	bool	触摸开关是否开启
AutoPlayStopEnabled	bool	智能播放是否开启
IsValid	bool	数据是否有效
ErrorMessage	str	错误信息 (如有)
Timestamp	datetime	时间戳

3.4 `DataReceivedEventArgs`

字段	类型	说明
EarSide	EarSide	EarSide.Left 或 EarSide.Right
Data	List[float]	EEG 数据点列表 (单位: 微伏 uV)
PacketCounter	int	包计数器
IsValid	bool	数据是否有效
ErrorMessage	str	错误信息 (如有)
Timestamp	datetime	时间戳

3.5 `SpectrumDataEventArgs`

能量谱数据，包含左右耳各频段能量。

字段	类型	说明
LeftDelta / RightDelta	float	Delta 波能量
LeftTheta / RightTheta	float	Theta 波能量
LeftAlpha / RightAlpha	float	Alpha 波能量
LeftBeta / RightBeta	float	Beta 波能量
LeftGamma / RightGamma	float	Gamma 波能量
LeftLowAlpha / RightLowAlpha	float	Low Alpha 能量
LeftHighAlpha / RightHighAlpha	float	High Alpha 能量
LeftLowBeta / RightLowBeta	float	Low Beta 能量
LeftHighBeta / RightHighBeta	float	High Beta 能量
LeftLowGamma / RightLowGamma	float	Low Gamma 能量
LeftHighGamma / RightHighGamma	float	High Gamma 能量
LeftSignalQuality	SignalQuality	左耳信号质量
RightSignalQuality	SignalQuality	右耳信号质量
Timestamp	datetime	时间戳

3.6 `MentalStateDataEventArgs`

脑状态反馈数据。

字段	类型	说明
Focus	float	专注度
Fatigue	float	疲劳度
Relax	float	放松度

Calm	float	平静度
Stress	float	压力值
Timestamp	datetime	时间戳

3.7 `ServerAuthResultEventArgs`

字段	类型	说明
IsSuccess	bool	认证是否成功
Permission	str	权限信息（成功时为权限列表，失败时为错误原因）
BoundAt	str	设备绑定时间（如有）
LastSeen	str	设备最近在线时间（如有）
ErrorMessage	str	详细错误信息（如有）
Timestamp	datetime	时间戳

4. 枚举定义

4.1 `ConnectionType`

成员	值	说明
BLE	0	蓝牙低功耗

4.2 `EarSide`

成员	值	说明
Left	0x00	左耳
Right	0x01	右耳

4.3 `SignalQuality`

成员	值	说明
OK	0	信号正常
SizeError	1	数据长度异常
OriginalMaxValue	2	原始数据最大值异常
OriginalZeroValue	3	原始数据零值异常
FilteredMaxValue	4	滤波后最大值异常
FilteredZeroValue	5	滤波后零值异常
ThresholdError	6	阈值异常
PowerFrequencyError	7	工频干扰异常
OtherError	8	其他异常

5. 辅助函数

5.1 `save\_eeg\_to\_excel`

将左右耳 EEG 数据保存为 Excel（若未安装 `openpyxl` 则自动回退为 CSV）。

```
def save_eeg_to_excel(
    left_data: List[float],
    right_data: List[float],
    filename: Optional[str] = None,
    metadata: Optional[Dict[str, Any]] = None
) -> str
```

参数	说明
left_data	左耳 EEG 数据列表
right_data	右耳 EEG 数据列表
filename	输出文件名，默认自动生成

metadata

可选元数据字典，会写入文件头部

返回值

实际保存的文件路径

6. 完整使用示例

6.1 搜索、连接与原始数据接收

```
import asyncio
import datetime
from naoyunsdk import (
    NaoyunSdkApi, EarSide, BleDeviceInfo,
    DeviceStateEventArgs, DataReceivedEventArgs
)

async def main():
    sdk = NaoyunSdkApi()
    sdk.Initialize("YOUR_APP_ID", "YOUR_APP_SECRET", is_domestic=True)

    # 设置回调
    sdk.BleDeviceDiscovered = lambda d: print(f"发现: {d.Name}")
    sdk.ConnectionStateChanged = lambda e: print(f"连接状态: {e.IsConnected}")
    sdk.DataReceived = lambda e: print(
        f"{'左耳' if e.EarSide == EarSide.Left else '右耳'}数据: {len(e.Data)} 点"
    )

    # 搜索并连接
    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if devices:
        await sdk.ConnectBleAsync(devices[0].Id)
        await sdk.SendStartDataAsync()
        await asyncio.sleep(10)
        await sdk.SendStopDataAsync()
        await sdk.DisconnectAsync()

asyncio.run(main())
```

6.2 启动脑状态任务

```
import asyncio
import datetime
```

```

from naoyunsdk import NaoyunSdkApi, MentalStateDataEventArgs

async def main():
    sdk = NaoyunSdkApi()
    sdk.Initialize("YOUR_APP_ID", "YOUR_APP_SECRET")

    sdk.MentalStateDataReceived = lambda e: print(
        f"专注={e.Focus:.1f} 疲劳={e.Fatigue:.1f} 放松={e.Relax:.1f}"
    )

    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if devices and await sdk.ConnectBleAsync(devices[0].Id):
        await sdk.SendStartDataAsync()
        sdk.StartMentalStateTask(interval_seconds=1.0)
        await asyncio.sleep(30)
        sdk.StopMentalStateTask()
        await sdk.SendStopDataAsync()
        await sdk.DisconnectAsync()

asyncio.run(main())

```

6.3 获取滤波后的 EEG 数据并保存

```

import asyncio
import datetime
from naoyunsdk import NaoyunSdkApi, EarSide, save_eeg_to_excel

async def main():
    sdk = NaoyunSdkApi()
    sdk.Initialize("YOUR_APP_ID", "YOUR_APP_SECRET")

    devices = await sdk.StartBleScanAsync(datetime.timedelta(seconds=5))
    if devices and await sdk.ConnectBleAsync(devices[0].Id):
        await sdk.SendStartDataAsync()
        await asyncio.sleep(10)

        # 获取原始数据与 1-78Hz 滤波数据
        left_raw = sdk.GetLatestEegData(EarSide.Left, 5, use_filtered_data=False)
        leftflt = sdk.GetLatestEegData(EarSide.Left, 5, use_filtered_data=True)
        rightflt = sdk.GetLatestEegData(EarSide.Right, 5, use_filtered_data=True)

        # 保存
        save_eeg_to_excel(leftflt, rightflt, metadata={"subject": "demo"})

        await sdk.SendStopDataAsync()
        await sdk.DisconnectAsync()

```

```
asyncio.run(main())
```
